CHAPTER 2 — EVOLUTION OF SOFTWARE ECONOMICS

2.1 SOFTWARE ECONOMICS

Introduction

Software engineering is dominated by **intellectual activities** that focus on solving problems of immense complexity, involving numerous unknowns and competing perspectives.

The software industry has evolved through different approaches:

- **1960s–1970s:** Software development was mainly considered **craftsmanship**.
- **1980s–1990s:** The industry transitioned toward a **software engineering discipline**.
- **2000 onwards:** The industry moved toward a more **production-intensive approach**, emphasizing automation and economies of scale.

The study of **software economics** helps in understanding the factors that influence software development **effort and cost**.

What is Software Economics?

Software economics deals with the economic aspects of software development, particularly the relationship between the factors that determine the **effort and cost** required to develop software.

According to the reference, most software cost models can be represented using **five basic parameters**:

1. **Size**
2. **Process**
3. **Personnel**
4. **Environment**
5. **Required Quality**

These parameters are used to understand and estimate the effort required for software development.

Why is Software Economics Important?

Software development is different from many traditional manufacturing activities because it involves:

- Complex intellectual work
- Numerous unknowns
- Human creativity
- Different perspectives
- Significant development effort
- Difficulty in accurately estimating cost and effort

A major economic issue in software development is **diseconomy of scale**.

Unlike most manufacturing processes:

The more software you build, the more expensive it is per unit item.

Therefore, understanding software economics is important for improving software development performance and achieving better **Return on Investment (ROI)**.

Five Basic Parameters of Software Economics

1. Size of the End Product

The **size of the end product** refers to the amount of software that has to be developed using human-generated components.
It is typically quantified using:

- **Number of source instructions**
- **Number of Function Points**

These measures represent the amount of functionality or software that needs to be developed.

2. Process Used to Produce the End Product

The **process** refers to the software development process used to produce the end product.
An important capability of the process is its ability to **avoid non-value-adding activities**, such as:

- **Rework**
- **Bureaucratic delays**
- **Communication overhead**

A better process helps reduce unnecessary effort and improves development efficiency.

3. Capabilities of Software Engineering Personnel

This parameter represents the **capabilities of the people involved in software development**.
It particularly includes:

- Experience with **computer science issues**
- Experience with the **application domain**
- Ability to handle the technical issues of the project

The capabilities and experience of software personnel have a direct influence on development effort.

4. Environment

The **environment** consists of the **tools and techniques** available to support efficient software development and to automate the process.
The development environment therefore helps in:

- Improving development efficiency
- Supporting software development activities
- Automating the software process

5. Required Quality

The **required quality** refers to the quality characteristics expected from the software product.
The reference specifically includes:

- **Features**
- **Performance**

- **Reliability**
- **Adaptability**

The required level of quality is one of the parameters affecting software development effort.

Relationship Between the Parameters and Effort

The reference gives the following general relationship:

Effort = (Personnel)(Environment)(Quality)(Size^Process)

This represents the relationship between the major software economics parameters and the **estimated development effort**.

Several parametric software cost models can generally be abstracted into this form.

Diseconomy of Scale

One of the important concepts in software economics is the **diseconomy of scale**.

Definition

Diseconomy of scale means that as the size of a software system increases, the **cost per unit of software also increases**.

The reference explains that this occurs because the **process exponent is greater than 1.0**.

Important Point

This is contrary to most manufacturing processes, where producing more units may reduce the cost per unit.

In software:

Larger software size → Higher effort → Higher cost per unit

Example

For a particular application:

100,000-line system

- Development effort = **900 staff-months**
- Productivity ≈ **111 lines/staff-month**
- Effort ≈ **1.37 hours/line**

10,000-line system

If all other parameters remain constant:

- Development effort = **62 staff-months**
- Productivity ≈ **175 lines/staff-month**
- Effort ≈ **0.87 hour/line**

Therefore:

10,000-line system → Lower cost per line

compared with

100,000-line system → Higher cost per line

This demonstrates the **diseconomy of scale in software development**.

Evolution of Software Economics

The reference identifies **three generations of software economics**.

Generation	Period	Major Characteristic
Conventional	1960s–1970s	Craftsmanship
Transition	1980s–1990s	Software engineering
Modern Practices	2000 and later	Software production

1. Conventional — 1960s–1970s

- Software development was considered **craftsmanship**.
- Organizations used **custom tools**.
- Processes were **custom/ad hoc**.
- Virtually all software components were custom-built.
- Project performance was highly unpredictable.
- Cost, schedule and quality objectives were almost always underachieved.

2. Transition — 1980s–1990s

- Software development became more of an **engineering discipline**.
- Organizations used **more-repeatable processes**.
- **Off-the-shelf tools** were used.
- More than **70%** of components were generally custom-built.
- Less than **30%** could be commercial/off-the-shelf components.
- Some organizations began achieving economies of scale.
- However, increasing application complexity created difficulties.

3. Modern Practices — 2000 and later

- Software development moved toward **software production**.
- Uses **managed and measured processes**.
- Uses **integrated automation environments**.
- Mostly uses **off-the-shelf components**.
- Approximately **70%** of components may be off-the-shelf.
- Approximately **30%** may need to be custom-built.
- Advances in software technology and integrated production environments allow component-based systems to be produced rapidly.

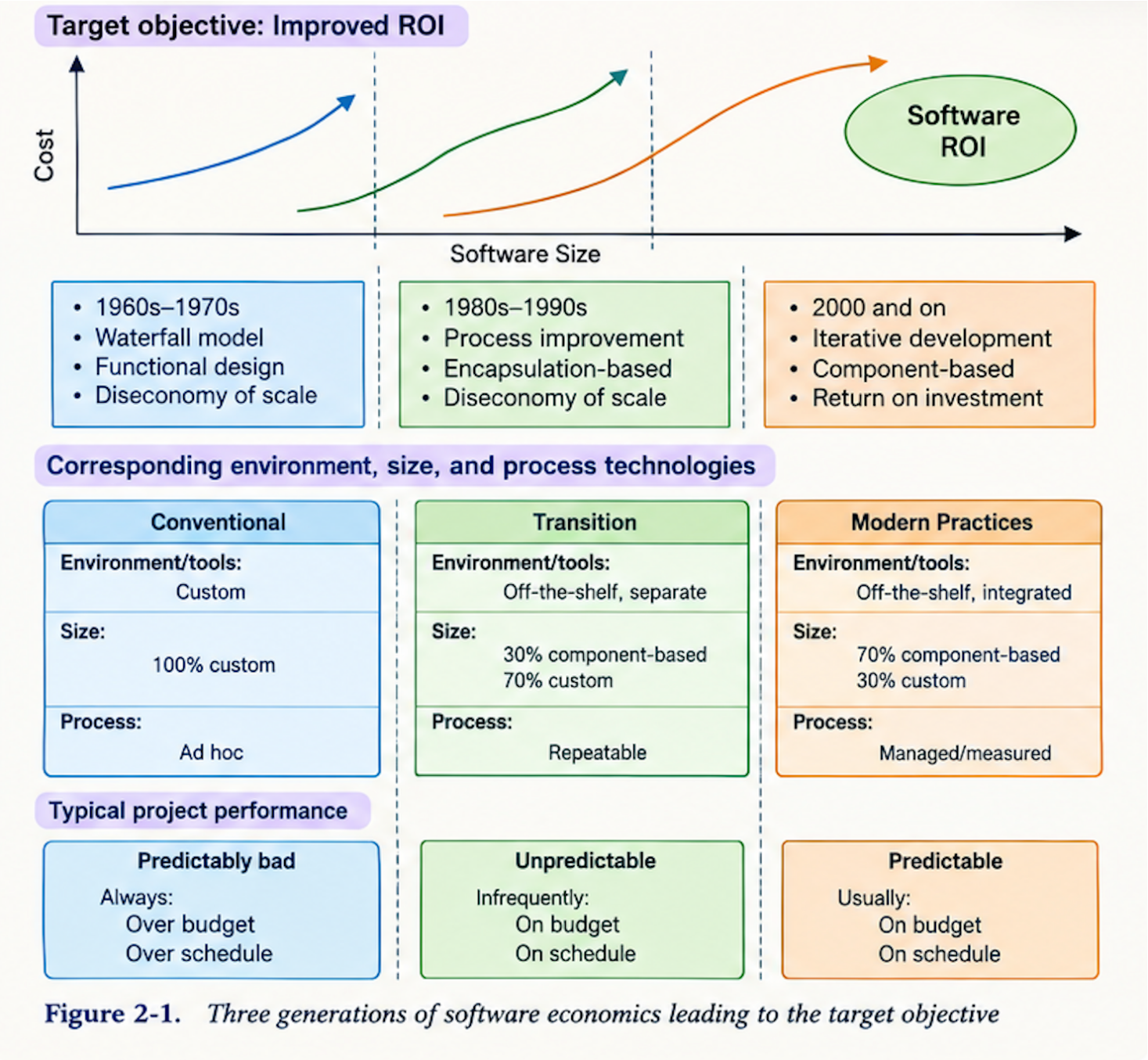


Figure 2-1 — Three Generations of Software Economics

The reference presents the evolution as:

Conventional

↓

Transition

↓

Modern Practices

↓

Target Objective: Improved ROI

The figure compares the generations based on:

- Software size
- Environment/tools

- Process
- Components
- Typical project performance

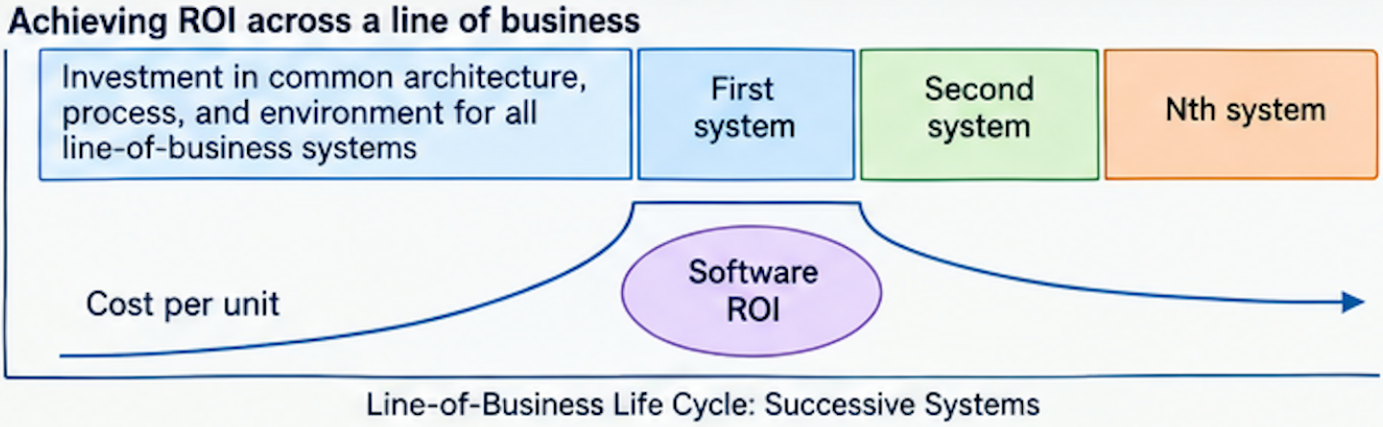


Figure 2-2. *Achieving ROI across a line of business*

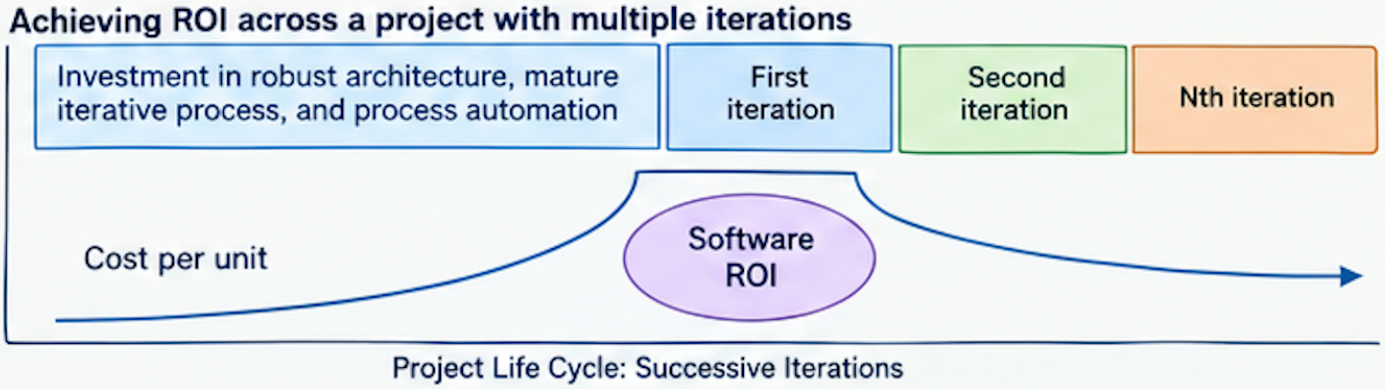


Figure 2-2. *Return on investment in different domains*

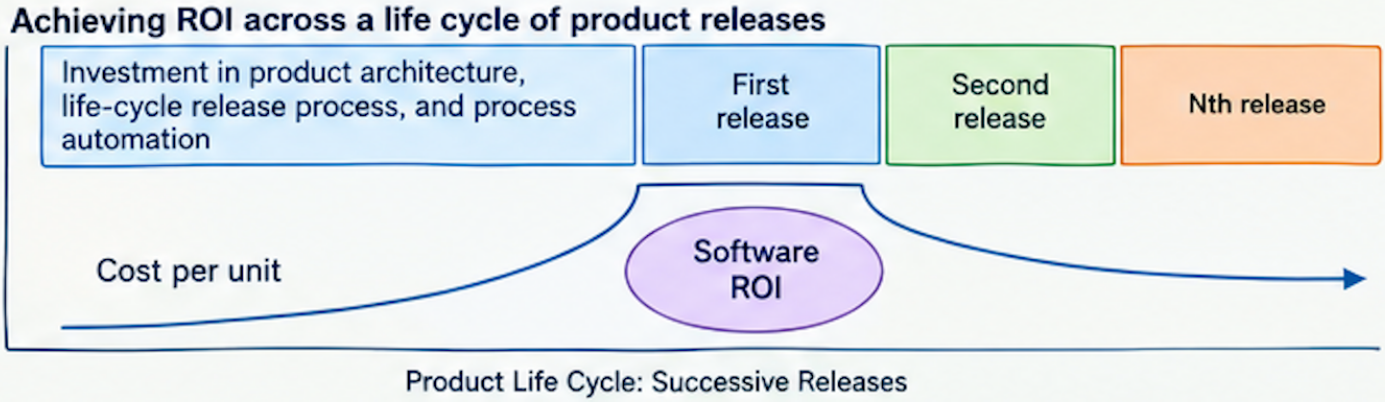


Figure 2-2. *Return on investment in different domains*

Key Points to Remember ⭐

1. Software economics is concerned with the **economic aspects of software development**, particularly effort and cost.
2. Software cost models are based primarily on **five parameters**:
    - Size
    - Process

    - Personnel
    - Environment
    - Required Quality
3. Software size is commonly quantified using **source instructions or Function Points**.
4. A good process avoids **rework, bureaucratic delays and communication overhead**.
5. Personnel capability includes experience with **computer science and application-domain issues**.
6. Environment consists of **tools and techniques** that support and automate development.
7. Required quality includes **features, performance, reliability and adaptability**.
8. The general effort relationship is:
   Effort = (Personnel)(Environment)(Quality)(Size^Process)
9. When the **process exponent > 1.0**, software development exhibits **diseconomy of scale**.
10. Software economics evolved through:
    Conventional → Transition → Modern Practices
11. The target objective of improved software economics is **improved ROI (Return on Investment)**

2.2 PRAGMATIC SOFTWARE COST ESTIMATION

Software cost estimation is a critical activity because the **cost, effort and schedule** of a software project must be estimated before development decisions are made.
The reference points out that estimating software cost is difficult, particularly because there are **few well-documented case studies of projects using modern iterative development approaches**.

2.2.1 Problems in Software Cost Estimation

A major problem is the lack of **consistent and comparable project data**.

Major difficulties

1. **Lack of well-documented case studies**
    - There are relatively few empirical databases containing successful modern iterative-development projects.
2. **Inconsistent software metrics**
    - The software industry has not consistently defined the basic units used for measurement.
3. **Difficulty in comparing project data**
    - It is difficult to collect homogeneous project data:
        - Within a single organization
        - Across different organizations
        - Across different processes
        - Across different programming languages
        - Across different application domains
4. **Different definitions of software size**
    - A basic unit such as a **Source Line of Code (SLOC)** or **Function Point (FP)** may be counted differently in different environments.

Important Point

The **exact definition** of a SLOC or Function Point is less important than ensuring that **everyone uses the same definition consistently**.

2.2.2 Three Major Debates in Software Cost Estimation

The reference identifies three long-standing topics of debate among developers and vendors of software cost-estimation models and tools:

1. Which cost estimation model to use

There are many different software cost-estimation models. The challenge is selecting the model that is most appropriate for a particular project.

2. How to measure software size

The two major approaches are:

- **Source Lines of Code (SLOC)**
- **Function Points (FP)**

3. What constitutes a good estimate

It is important to determine what characteristics make a software cost estimate **credible and useful**.

2.2.3 Popular Software Cost Estimation Models

The reference mentions several popular cost-estimation models:

- **COCOMO**
- **CHECKPOINT**
- **ESTIMACS**
- **KnowledgePlan**
- **Price-S**
- **ProQMS**
- **SEER**
- **SLIM**
- **SOFTCOST**
- **SPRQ/20**

There are also numerous **organization-specific models**.

Among these, **COCOMO and its successors** receive particular emphasis in the reference.

COCOMO

COCOMO is one of the most open and well-documented software cost-estimation models.

The reference uses COCOMO and **COCOMO II** as important bases for discussing software economics.

2.2.4 Measurement of Software Size

There are two major objective approaches to measuring software size:

1. Source Lines of Code (SLOC)

SLOC measures software size in terms of the number of source-code lines/instructions.

2. Function Points (FP)

Function points measure software size based on the **functionality provided by the software**.
The reference states that both approaches have proved more valuable than a **subjective or ad hoc measurement** of software size.

SLOC as a Measure of Size

SLOC has traditionally been used as a measure of software size.
For example, describing a program as:
1,000 source lines of code
gives a general indication of its size.
However, simply describing software using function points or other measures can sometimes require additional clarification to understand the actual software being developed.

Earlier usefulness of SLOC

SLOC worked particularly well when applications were:

- Predominantly custom-built
- Developed using traditional programming approaches

SLOC measurement was also relatively easy to **automate and instrument**.

Problems with SLOC

Modern software development has introduced:

- Higher-level programming languages
- Components
- Automatic source-code generation
- Object-oriented development
- Reuse

These developments have made SLOC a **more ambiguous measure**.
Therefore, accurately counting SLOC can become difficult in modern software projects.

2.2.5 Function Points

Function Points (FP) are another important approach for measuring software size.
Function points have a large following in the software industry.
The **International Function Point Users Group (IFPUG)**, formed in **1984**, is identified in the reference as the dominant software measurement association in the industry.

Advantages of Function Points

The primary advantage is that function points are:

- **Independent of technology**
- A useful primitive unit for **comparison among projects and organizations**

Thus, function points can provide a more technology-independent basis for software-size comparisons.

Disadvantages of Function Points

The main disadvantage is that:

- Primitive definitions are **abstract**
- Measurements are **not easily derived directly from evolving software artifacts**

SLOC vs Function Points

Aspect	SLOC	Function Points
Basic basis	Source code	Functionality
Technology dependence	More technology-dependent	Technology-independent
Traditional usefulness	High for custom-built software	Useful for cross-project comparison
Modern problem	Becomes ambiguous with reuse, generators and object orientation	Definitions are abstract
Measurement	Relatively easy to automate	Not easily derived directly from evolving artifacts
Main advantage	Concrete source-code measure	Better technology-independent comparison

Important Conclusion

Both measures have drawbacks, but the reference emphasizes:

Using some measure is better than using none at all.

For **cross-project or cross-organization comparisons**, function points are recommended in the reference as the preferred measure.

Function points can also be a **more accurate estimator during the early phases** of the project life cycle.

In later phases, **SLOC can become a more useful and precise measurement basis** for various metrics perspectives.

2.2.6 Accuracy of Conventional Cost Models

The general accuracy of conventional cost models such as **COCOMO** has been described as:

"Within 20% of actuals, 70% of the time."

This indicates that conventional software cost estimation contains a significant degree of **unpredictability**.

2.2.7 Why Software Estimates Are Difficult to Beat

In conventional software development:

- Projects rarely perform significantly better than planned.
- Unless specific incentives exist for beating the schedule, teams rarely finish earlier than planned.
- Teams and individuals may redistribute effort to meet schedule or quality objectives.

The reference emphasizes that the conventional software-development process has considerable uncertainty, making accurate estimation difficult.

2.2.8 Predominant Cost Estimation Process

The reference presents the **predominant cost estimation process** in Figure 2-3.

Basic process

Software Manager / Architecture Manager / Development Manager / Assessment Manager
↓
Determine target cost
↓
Cost Modelers
↓
Cost Estimate
↓
Risks, Options, Trade-offs, Alternatives

The software project manager first establishes a **target cost** and then adjusts/manipulates parameters until the target cost can be justified.

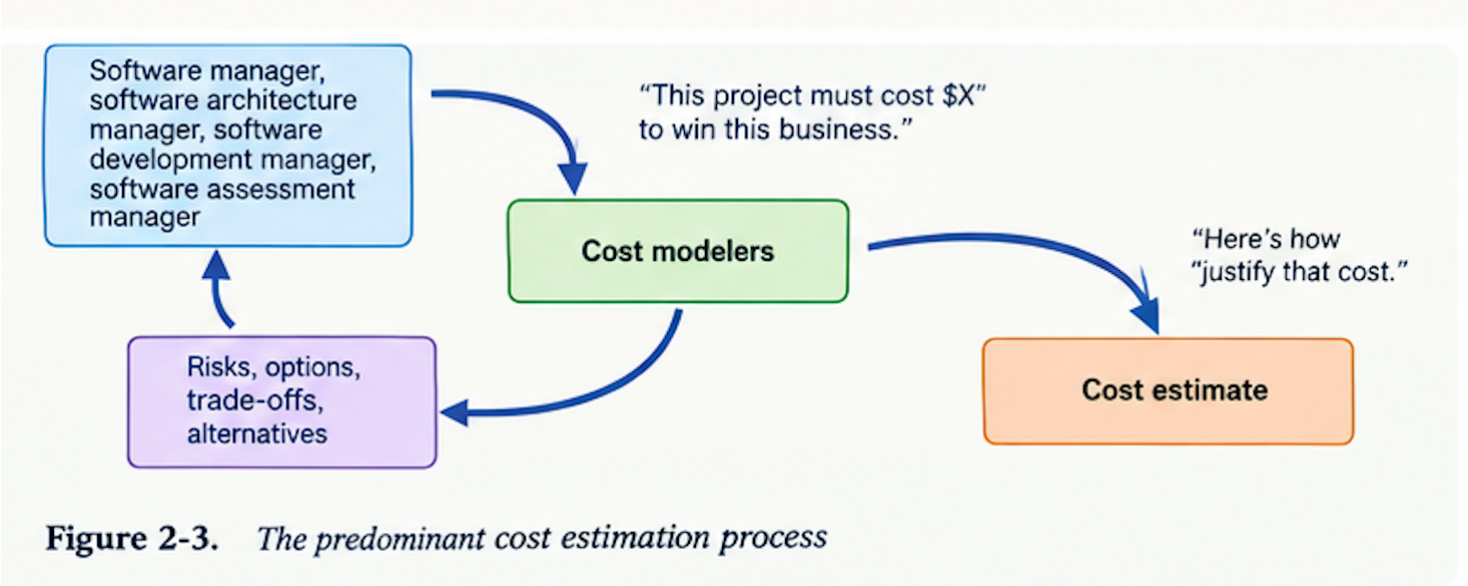


Figure 2-3. *The predominant cost estimation process*

Figure 2-3 — The Predominant Cost Estimation Process

2.2.9 Bottom-Up vs Top-Down Cost Estimation

Most real-world use of cost models is **bottom-up in practice**, although it may actually involve:

Target-cost approach

The project manager:

1. Defines the **target cost**.
2. Sizes the software.
3. Manipulates the model parameters.
4. Continues until the target cost can be justified.

Reasons for establishing a target cost

A target cost may be established to:

- Win a proposal
- Obtain customer funding
- Obtain internal corporate funding
- Achieve another organizational goal

2.2.10 Risks, Options, Trade-offs and Alternatives

The target-cost approach is **not necessarily bad**.

It can be useful because it forces the project manager to:

- Analyze cost risks
- Understand sensitivities
- Examine trade-offs
- Consider alternative plans
- Discuss cost information with stakeholders

The process may lead to changes in:

- Plans
- Designs
- Processes
- Project scope

Thus, it can provide a useful basis for **cost estimation and overall cost analysis**.

2.2.11 Independent Cost Estimates

A practical lesson from the field is that **independent cost estimates**, prepared by people independent of the development team, are usually inaccurate.

A credible estimate should instead be produced by a **competent project team**.

The reference identifies the following team members:

- **Software project manager**
- **Software architect**
- **Development manager**
- **Test manager**

This team should:

1. Produce estimates.
2. Iterate through several estimates.
3. Perform sensitivity analysis.
4. Take ownership of the final cost estimate.
5. Work toward project success.

2.2.12 Attributes of a Good Software Cost Estimate

A good software cost estimate should have the following characteristics:

1. Supported by the Project Team

It should be **conceived and supported by**:

- Project manager
- Architecture team
- Development team
- Test team

The team should be accountable for performing the work.

2. Accepted by Stakeholders

It should be:

- **Ambitious**
- **Realizable**
- Accepted by all stakeholders.

3. Based on a Well-Defined Cost Model

The estimate should use:

- A well-defined software cost model
- A credible basis for estimation

4. Based on Relevant Project Experience

It should use a database of relevant project experience containing similarities in:

- Processes
- Technologies
- Environments
- Quality requirements
- People

5. Detailed Risk Understanding

The estimate should contain enough detail so that:

- Key risk areas are understood.
- Probability of success can be objectively assessed.

2.2.13 Ideal Estimate

An **ideal estimate** would be derived from a **mature cost model** and an experience base containing multiple similar projects.

Ideally, those projects should have been:

- Performed by the **same team**
- Using the **same mature processes**
- Using the **same tools**

Such a situation is rare when a team starts a completely new project.

However, good estimates can be achieved in the **later life-cycle phases of a mature project** using a mature process.

⭐ Exam Focus

Remember these points:

- Software cost estimation is difficult because of **limited empirical data and inconsistent metrics**.
- Three major debates:
    1. Which cost model to use
    2. SLOC vs Function Points
    3. What constitutes a good estimate
- Important cost models include **COCOMO, CHECKPOINT, ESTIMACS, KnowledgePlan, Price-S, ProQMS, SEER, SLIM, SOFTCOST and SPRQ/20**.
- Two major software-size measures are **SLOC and Function Points**.
- SLOC becomes more ambiguous with **reuse, components, automatic code generation and object-oriented development**.
- Function Points are **technology-independent** and useful for comparisons.
- Conventional cost-model accuracy: **within 20% of actuals, 70% of the time**.
- Most real-world cost-model usage involves a **target-cost approach**.
- A credible estimate should involve the **project manager, architect, development manager and test manager**.
- A good estimate should be **realistic, stakeholder-accepted, model-based, experience-based and detailed enough to understand risks**.
- An ideal estimate comes from **mature models + similar project experience + same team/process/tools**.

CHAPTER 3 — IMPROVING SOFTWARE ECONOMICS

3.1 REDUCING SOFTWARE PRODUCT SIZE

The most significant way to improve **affordability and Return on Investment (ROI)** is usually to produce a product that achieves the design goals with the **minimum amount of human-generated source material**.

Component-based development is used as a general approach for reducing the amount of source language required to achieve a software solution.

Major approaches for reducing software product size include:

1. **Higher-order programming languages**
2. **Object-oriented methods and visual modeling**
3. **Reuse**
4. **Commercial components**
5. **Automatic code production**

These approaches aim to achieve a required system with fewer lines of human-specified source directives/statements.

3.1.1 LANGUAGES

Universal Function Points (UFPs) are useful for making **language-independent estimates early in the life cycle**.

The basic units of function points include:

- External user inputs
- External outputs
- Internal logical data groups
- External data interfaces
- External inquiries

SLOC (Source Lines of Code) metrics become useful after a candidate solution has been formulated and the implementation language is known.

There is substantial data relating **SLOC to Function Points**. These data illustrate the relative expressiveness of different programming languages.

Language Expressiveness

The reference gives the following examples:

Language	Approximate SLOC
Assembly language	1,000,000
C	400,000
Ada 83	220,000
Ada 95 or C++	175,000

These values indicate the **relative expressiveness** provided by different languages.

Important Observation

For the **same functionality**, a more expressive language can require substantially fewer source lines.

Therefore:

Higher language expressiveness → Fewer source lines → Smaller development effort → Potentially lower cost

However, these data must be interpreted carefully because they span different **application domains, corporations and technology generations**.

Example from the Reference

For a fixed amount of functionality, the reference illustrates the difference using approximately:

- **1,000,000 lines** of Assembly language
- **400,000 lines** of C
- **220,000 lines** of Ada 83
- **175,000 lines** of Ada 95 or C++

Ada 83 and Ada 95

The development of **Ada 83** was partly motivated by the desire to increase software expressiveness.

Other objectives included:

- Reliability
- Support for real-time programming
- Maintainability

- Improved ROI through language standardization

Ada incorporated several software-engineering advances such as:

- Language-enforced configuration control
- Separation of interface and implementation
- Architectural control primitives
- Encapsulation
- Concurrency support

Ada 95 was a planned language upgrade that incorporated new technology and lessons learned from field applications.

The difference in expressiveness between Ada 83 and Ada 95 was mainly due to features added to support **object-oriented programming**.

First-order estimation: Object-oriented programming can allow programs to be written in about **30% fewer source lines**.

C and C++

C++ incorporated several advances found in Ada and provided advanced support for **object-oriented programming**.

However, C++ retained compatibility with C.

Important observation

The compatibility made it easier for C programmers to transition to C++, but it also created a tendency for some programmers to use a **C programming mindset** while using a C++ compiler. This can prevent programmers from taking full advantage of object-oriented features.

3.1.2 OBJECT-ORIENTED METHODS AND VISUAL MODELING

There was a widespread movement toward **object-oriented technology** during the 1990s.

Studies indicated that object-oriented programming languages could improve:

- Software productivity
- Software quality

However, the economic benefit was difficult to demonstrate because of the **high cost of training** in object-oriented design methods such as the **Unified Modeling Language (UML)**.

Role in Reducing Software Size

Object-oriented technology provides more-formalized notations for:

- Capturing software abstractions
- Visualizing software abstractions
- Representing the problem and its solution

Its fundamental impact is therefore in **reducing the overall size of what needs to be developed**.

Three Important Reasons Object-Oriented Projects Can Succeed

The reference describes three reasons given by Booch.

1. Common Vocabulary

An object-oriented model of the problem and its solution encourages a **common vocabulary**

between end users and developers.
This creates a shared understanding of the problem being solved.
Economic relationship:
Object-oriented technology → Common understanding → Better teamwork and interpersonal communication

2. Continuous Integration

The use of **continuous integration** provides opportunities to:

- Recognize risks early
- Make incremental corrections
- Avoid destabilizing the entire development effort

It supports an **architecture-first process**, in which integration becomes an early and continuous life-cycle activity.

3. Separation of Concerns

An object-oriented architecture provides a clear **separation of concerns** among different elements of a system.
This creates boundaries that prevent a change in one part of the system from damaging the entire architecture.
This feature is important because of the languages and environments available for implementing object-oriented architectures.

Five Characteristics of a Successful Object-Oriented Project

Booch summarized the following characteristics:

1. **Ruthless focus on development of a system** that provides a well-understood collection of essential minimal characteristics.
2. A **results-centered culture** that encourages communication and is not afraid to fail.
3. **Effective use of object-oriented modeling.**
4. A **strong architectural vision.**
5. A **well-managed iterative and incremental development life cycle.**

Important Point

These characteristics are not exclusively related to object orientation.
However, **object-oriented methods, notations and visual modeling provide strong technology support for the overall process framework.**

Effect of Commercial Components and Automatic Code Generation

Commercial components and automatic code generators such as:

- **CASE tools**
- **GUI builders**

can further reduce the amount of **human-generated source code**.
This reduces:

Source-code size → Team size → Development time
For example, integrating:

- A commercial **Database Management System (DBMS)**
- A commercial **GUI builder**
- Commercial **middleware**

could reduce the effective development size to approximately:
75,000 lines of Ada 95 or C++ + integration of several commercial components

Economic Impact of Reducing Software Size

The difference between large and small projects has a **greater-than-linear impact on life-cycle cost**.
Therefore, using:

- The highest appropriate-level language
- Appropriate commercial components
- Effective abstraction technologies

can have a large potential impact on project cost.

Benefits of reducing size

Reducing software size usually improves:

- **Understandability**
- **Changeability**
- **Reliability**

Possible disadvantage

Higher-level abstraction technologies can degrade performance by increasing consumption of:

- Processor cycles
- Memory
- Communication bandwidth

Many of these disadvantages can be overcome through:

- Hardware performance improvements
- Optimization techniques

However, such improvements are **far less effective in embedded platforms**.

3.1.3 REUSE

Reuse means using existing software components, knowledge, architectures, processes or other existing artifacts instead of developing everything from scratch.
Reuse has been part of software engineering since the early days of programming languages.
It helps minimize development costs while achieving the required:

- Performance
- Feature set
- Quality

Important Point

Reuse is often treated as a technology, but the reference emphasizes that it is better understood

as an important **discipline affecting the efficiency of workflows and quality of artifacts**.
It is closely related to **Return on Investment (ROI)**.

Major Obstacles to Reuse

One of the biggest obstacles to reuse has been the lack of standards across:

- Operating systems
- Notations
- Machine architectures
- Tools
- Languages
- Standards

A counterexample mentioned in the reference is the level of reuse made possible by Microsoft's success on the PC platform.

Economic Motivation for Reuse

In general, things get reused for **economic reasons**.
A key measure of whether a component is truly reusable is whether an organization can make money from it.
Commercially supported reusable components generally have:

1. **Economic motivation** for continued support.
2. Ownership for improving quality, adding features and adopting new technologies.
3. A sufficiently broad customer base to remain profitable.

Cost of Developing Reusable Components

Developing a reusable component is **not trivial**.
The initial investment can be high, making it difficult to establish a convincing business case unless the component is intended for significant reuse or commercial sale.
For commercial reusable components, an organization generally needs:

- Development group
- Support infrastructure
- Product-oriented sales and marketing infrastructure

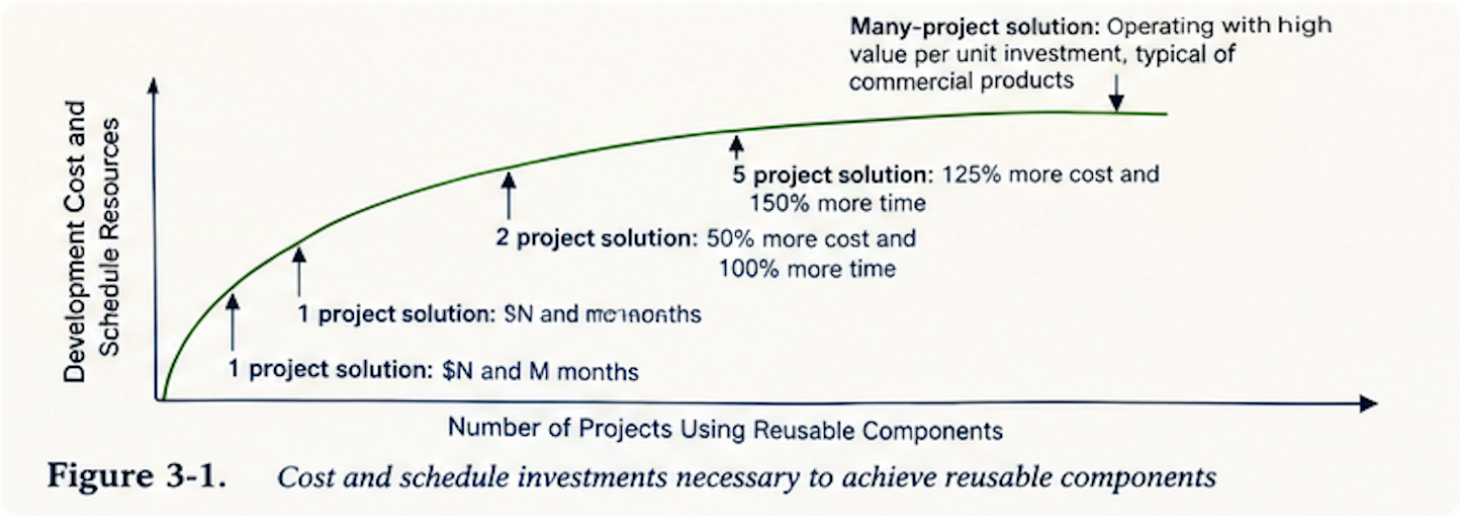


Figure 3-1. *Cost and schedule investments necessary to achieve reusable components*

Important Conclusion on Reuse

Although the value of reuse can be immense, the reference notes that there have been relatively few major success stories in software component reuse outside commercial products such as:

- Operating systems
- Database management systems
- Middleware
- Networking
- GUI builders
- Office applications

At the same time, practically every successful software project probably exploits some form of reuse.

3.1.4 COMMERCIAL COMPONENTS

A common approach in many domains is to maximize the integration of **commercial components and off-the-shelf products**.

The main objective is to reduce **custom development**.

However, using commercial components is **not always straightforward**.

The choice between commercial components and custom development can significantly affect:

- Architecture
- Quality
- Cost
- Supportability

Therefore, these decisions should be made **early in the life cycle as part of architectural design**.

Commercial Components vs Custom Development

Commercial Components	Custom Development
Predictable license costs	Complete change freedom

Broadly used, mature technology	Smaller, often simpler implementations
Available immediately	Often better performance
Dedicated support organization	Control over development and enhancement
Hardware/software independence	—
Rich in functionality	—
Frequent upgrades can be a disadvantage	Expensive and unpredictable development
Up-front license fees	Unpredictable availability date
Recurring maintenance fees	Undefined maintenance model
Dependency on vendor	Often immature and fragile
Run-time efficiency sacrifices	Single-platform dependency
Functionality constraints	Drain on expert resources
Integration is not always trivial	—
No control over upgrades and maintenance	—
Unnecessary features may consume resources	—
Often inadequate reliability/ stability	—
Multiple-vendor incompatibilities	—

⭐ EXAM FOCUS — 3.1 REDUCING SOFTWARE PRODUCT SIZE

Remember the central idea:

Reduce the amount of human-generated source material required to achieve the required functionality.

Four major approaches

Higher-order languages

↓

Object-oriented methods & visual modeling

↓
Reuse
↓
Commercial components / automatic code generation

Key economic chain

Smaller software size
→ Less development effort
→ Smaller teams / less development time
→ Lower life-cycle cost
→ Better affordability and ROI

Important numbers to remember

Language	Approx. SLOC
Assembly	1,000,000
C	400,000
Ada 83	220,000
Ada 95 / C++	175,000
With commercial components	75,000 lines + component integration

CHAPTER 3 — IMPROVING SOFTWARE ECONOMICS

3.2 IMPROVING SOFTWARE PROCESSES

Software development requires a complex combination of **sequential and parallel activities**. As project size increases, additional overhead is required to manage this complexity.
The objective of process improvement is to **maximize the allocation of resources to productive activities and minimize the impact of overhead activities** on personnel, computers, and schedule.

3.2.1 THREE LEVELS OF SOFTWARE PROCESS

The reference identifies **three levels of process**:

1. **Metaprocess**
2. **Macroprocess**
3. **Microprocess**

These levels overlap to some extent, but they have different objectives, audiences, metrics, concerns and time scales.

Table 3-4 — Three Levels of Process and Their Attributes

Attribute	Metaprocess	Macroprocess	Microprocess
Subject	Line of business	Project	Iteration
Objectives	Line-of-business profitability; competitiveness	Project profitability; risk	Resource management
Budget / Schedule / Quality	Project budget, schedule, quality	Milestone budget, schedule, quality	—
Audience	Acquisition authorities, customers, organizational management	Software project managers, software engineers	Subproject managers, software engineers
Metrics	Project predictability; market share	On budget, on schedule; project scrap and rework	On budget, on schedule; major milestone progress; release/iteration scrap and rework
Concerns	Bureaucracy vs. standardization	Quality vs. financial performance	Content vs. schedule
Time Scale	6–12 months	1 to many years	1–6 months

Important: The **macroprocess** is the project-level process that affects the software cost estimation model discussed in this chapter.

3.2.2 PRODUCTIVE AND OVERHEAD ACTIVITIES

All software processes consist of two broad categories of activities:

1. Productive Activities

These activities result in **tangible progress toward the end product**.
Examples:

- Prototyping
- Modeling
- Coding
- Debugging
- User documentation

2. Overhead Activities

These activities have an **intangible impact on the end product**, but are necessary for managing the project.
Examples include:

- Plan preparation
- Documentation
- Progress monitoring
- Risk assessment
- Financial assessment
- Configuration control
- Quality assessment
- Integration
- Testing
- Late scrap and rework
- Management
- Personnel training
- Business administration

Main Objective

Although many overhead activities are value-adding, generally:
Less unnecessary overhead → More resources available for productive activities
Therefore, process improvement aims to:
Maximize productive activities and minimize the impact of overhead activities.

3.2.3 SCRAP AND REWORK

Scrap and rework are important sources of wasted effort in software development.

Early Scrap and Rework

Some early scrap and rework is a **natural and necessary part of prototyping**.
It helps the development team resolve the large number of possibilities in the solution space.

Late Scrap and Rework

Late scrap and rework is **clearly undesirable** because it occurs after considerable development effort has already been invested.
A good software process should minimize late scrap and rework.

Example

Suppose a team performs:

Analysis → Design → Coding → Testing
During testing, a design flaw is discovered.
The team may now need:
Redesign → Recode → Retest
Thus, one defect introduces an additional **sequential set of activities**.
These additional sequences are a major obstacle to **schedule compression**.

3.2.4 PERSONNEL TRAINING AS AN OVERHEAD ACTIVITY

The reference considers personnel training from the perspective of project economics.
Training is considered an **organizational responsibility rather than a project responsibility**.
A project manager who has to train personnel in:

- Processes
- Technologies
- Tools

is at a disadvantage compared with a project manager who receives a **fully trained workforce**.

Important Point

Staffing every project with completely trained personnel may not always be possible.
However:
Employing trained people is better than employing untrained people, other things being equal.
Therefore, training is not considered a **value-added project activity** in this context.

3.2.5 EFFECT OF PROCESS QUALITY ON COST AND SCHEDULE

The quality of the software process strongly affects:

- Required effort
- Development schedule
- Overall project cost
- Time required to achieve the product vision with required quality

The reference states that the difference between a **good process and a bad process** can affect overall cost estimates by approximately **50% to 100%**.
A better process can also have an even greater effect on reducing the time required to achieve the required product.

3.2.6 THREE DIMENSIONS OF SCHEDULE IMPROVEMENT

Schedule improvement can be achieved in **three different ways**.

1. Improve the Efficiency of Each Step

Take an **N-step process** and make every individual step more efficient.
N steps → More efficient N steps

2. Eliminate Some Steps

Take an **N-step process** and eliminate unnecessary steps.

The process becomes an **M-step process**, where:

M < N

Thus:

N-step process → Remove steps → M-step process

3. Increase Concurrency

Perform more activities simultaneously by increasing:

- Concurrency of activities
- Concurrency of resources

Thus, activities that would otherwise occur sequentially can be performed in parallel where appropriate.

Comparison of the Three Dimensions

Dimension	Main Idea
1. Improve efficiency	Make existing steps faster/better
2. Eliminate steps	Remove unnecessary process steps
3. Increase concurrency	Perform activities/resources in parallel

Most Important Point

Many organizational time-to-market strategies concentrate mainly on **improving the efficiency of individual steps**.

However, the process improvements emphasized in the reference focus more on:

- **Eliminating unnecessary steps**
- **Increasing concurrency**

because these provide greater potential for improvement.

3.2.7 PRIMARY FOCUS OF PROCESS IMPROVEMENT

The primary focus should be:

1. Achieving an **adequate solution in the minimum number of iterations**.
2. Eliminating as much **downstream scrap and rework** as possible.

Why?

Every instance of rework creates another **sequential chain of tasks**.

For example:

Analysis → Design → Coding → Testing

If a design flaw is found:

Redesign → Recode → Retest

Therefore, reducing rework directly helps in **compressing the development schedule**.

3.2.8 PROCESS IMPROVEMENT IN AN IMPERFECT WORLD

In a perfect software engineering environment, we might have:

- An accurate problem description
- An obvious solution space
- An experienced development team
- Adequate resources
- Stakeholders with common goals

Such an environment could potentially allow development in **one iteration with almost no scrap and rework**.

In reality, software projects operate in an **imperfect environment**.

Therefore, engineering activities must be managed so that the **scrap and rework profile does not negatively affect the win conditions of stakeholders**.

This should be the underlying premise for most process improvements.

⭐ EXAM FOCUS

Remember the central idea:

Process Improvement

→ Maximize productive activities

→ Minimize unnecessary overhead

→ Reduce late scrap & rework

→ Reduce sequential rework cycles

→ Compress schedule

→ Reduce effort and cost

Three ways to improve schedule

Improve existing steps

↓

Eliminate unnecessary steps

↓

Increase concurrency

Most important for this chapter

Minimum iterations + minimum downstream scrap and rework = major objective of process improvement.

3.3 IMPROVING TEAM EFFECTIVENESS

Introduction

Team effectiveness is one of the most important factors affecting software productivity and project success. The reference emphasizes that **teamwork is much more important than the**

sum of the individuals.
A project manager should therefore create a **balanced team** consisting of solid talent and highly skilled people in important leverage positions. (SVREC)

3.3.1 TEAM MANAGEMENT

Some important maxims of team management are:

1. **A well-managed project can succeed with a nominal engineering team.**
2. **A mismanaged project will almost never succeed, even with an expert team of engineers.**
3. **A well-architected system can be built by a nominal team of software builders.**
4. **A poorly architected system will flounder even with an expert team of builders.** (Shrimati Indira Gandhi College)

Key Idea

The success of a software project does not depend only on having highly talented individuals.
The **management of the team, team balance, job matching, and system architecture** are equally important.
Teamwork is much more important than the sum of the individuals.
The project manager must configure a balance between **solid talent** and **highly skilled people in leverage positions**. (SVREC)

3.3.2 BOEHM'S FIVE STAFFING PRINCIPLES

Boehm proposed the following five principles for staffing a software project:

1. The Principle of Top Talent

Use better and fewer people.
A small number of highly capable people can often contribute more effectively than a large number of less-skilled people.
However, this principle must be applied carefully because an excessively small or excessively large team can negatively affect team dynamics.

2. The Principle of Job Matching

Fit the tasks to the skills and motivation of the people available.
Different people have different skills and motivations.
Therefore, the project manager should assign people to tasks that match their capabilities.
Example:

- A good programmer may not necessarily be a good architect.
- A good architect may not necessarily be a good manager.
- A technically skilled person should be placed where that skill provides the greatest benefit.

Correct **person–job matching** improves productivity and team effectiveness. (TechBlogMU)

3. The Principle of Career Progression

An organization does best in the long run by helping its people to self-actualize.
Organizations should provide opportunities for employees to develop their skills and careers.
Training and development should help personnel become more capable and effective.
The reference distinguishes between:

- **Organizational training** → strategic and educational in nature.
- **Project training** → tactical and intended to provide skills immediately useful to the project.

4. The Principle of Team Balance

Select people who will complement and harmonize with one another.
Team balance is one of the most important factors in successful teamwork.
A team must have complementary abilities rather than simply a collection of highly talented individuals.
Important dimensions of team balance include:

Dimension	Examples
Raw skills	Intelligence, objectivity, creativity, organization, analytical thinking
Psychological makeup	Leaders/followers, risk takers/ conservatives, visionaries/ nitpickers, cynics/optimists
Objectives	Financial goals, feature set, quality, timeliness

An imbalance in any important dimension can make the project vulnerable. (Studocu)

5. The Principle of Phaseout

Keeping a misfit on the team doesn't benefit anyone.
A person who is unsuitable for the team can:

- Demotivate other team members.
- Reduce team effectiveness.
- Disturb team balance.
- Prevent better utilization of resources.
- Fail to develop or **self-actualize**.

Therefore, a persistent **misfit** should not simply be retained indefinitely.

3.3.3 IMPORTANCE OF TEAM BALANCE

The reference emphasizes that software development is essentially a **team sport**.
Managers should encourage a culture focused on:
Teamwork + Results
rather than:

Individual accomplishment

Among Boehm's five staffing principles:

- **Team balance** and **job matching** are the primary objectives.
- **Top talent** and **phaseout** are secondary objectives because they must be considered within the context of team balance.
- Career progression should be addressed, but it should not be allowed to take priority over the success of the team. (TechBlogMU)

3.3.4 LEADERSHIP QUALITIES OF SOFTWARE PROJECT MANAGERS

Software project managers require several leadership qualities to improve team effectiveness.

1. Hiring Skills

Few decisions are as important as **hiring decisions**.

The manager must place:

Right person → Right job

Although this sounds simple, achieving it is difficult in practice.

2. Customer-Interface Skill

The project manager must maintain good relationships with customers and stakeholders.

Avoiding **adversarial relationships among stakeholders** is an important prerequisite for project success.

3. Decision-Making Skill

A successful project manager must be capable of making effective decisions.

Software projects involve numerous decisions concerning:

- People
- Priorities
- Technical issues
- Resources
- Project direction

4. Team-Building Skill

The manager must:

- Motivate team members.
- Encourage progress.
- Make effective use of individual strengths.
- Help people become better performers.
- Remove misfits.
- Bring individual efforts into a common **team direction**.

5. Selling Skill

A project manager must be able to convince stakeholders about:

- Project decisions.

- Project priorities.
- Job positions.
- Changes to the existing approach.
- Achievements against objectives.

Selling requires continuous:

Negotiation + Compromise + Empathy

(www.slideshare.net)

⭐ EXAM FOCUS

Remember these 5 Boehm Staffing Principles

T → J → C → T → P

1. **Top Talent** → Use better and fewer people.
2. **Job Matching** → Match tasks with skills and motivation.
3. **Career Progression** → Help people self-actualize.
4. **Team Balance** → Select complementary people.
5. **Phaseout** → Remove/avoid misfits.

Most important points

- **Teamwork > Sum of individuals**
- Balance and coverage are critical.
- **Team balance + job matching** are primary objectives.
- Good management can make a nominal team successful.
- Poor management can make even an expert team fail.
- A good project manager needs **hiring, customer-interface, decision-making, team-building and selling skills**.

3.4 IMPROVING AUTOMATION THROUGH SOFTWARE ENVIRONMENTS

Introduction

The **tools and environment** used in the software process generally have a **linear effect on process productivity**.

Modern software development requires an integrated environment that supports **automation of the development process**. Tools reduce manual effort, improve consistency, support iterative development, and help development teams work with a smaller team more effectively.

3.4.1 SOFTWARE DEVELOPMENT ENVIRONMENT

The software environment provides automation support for evolving software engineering artifacts.

Important tools include:

- **Planning tools**
- **Requirements management tools**
- **Visual modeling tools**

- **Compilers**
- **Editors**
- **Debuggers**
- **Quality assurance analysis tools**
- **Test tools**
- **User interfaces**
- **Configuration management environments**

Among these, **configuration management environments provide the foundation for executing and instrumenting the process.**

Effect of Tools and Automation

The isolated impact of tools and automation can generally provide:

20% – 40% improvement in effort

However, tools and environments should not be viewed merely as individual productivity tools. They should be considered the **primary delivery vehicle for process automation and improvement**. Therefore, their overall impact can be much higher.

3.4.2 ROLE OF AUTOMATION IN PROCESS IMPROVEMENT

Section 3.2 focused mainly on process improvements that:

- Reduce **scrap and rework**
- Eliminate unnecessary process steps
- Minimize the number of iterations

Another important form of process improvement is:

Increasing the efficiency of individual process steps.

This is one of the primary contributions of the software environment.

Automation helps to:

- Automate manual tasks.
- Improve development efficiency.
- Provide rapid turnaround for iterative development.
- Allow development teams to iterate more freely.
- Maintain consistency among engineering artifacts.
- Support faster development and maintenance.

3.4.3 DEVELOPMENT AND MAINTENANCE ENVIRONMENT AS A FIRST-CLASS ARTIFACT

An important emphasis of the modern approach is to define the **development and maintenance environment as a first-class artifact of the process**.

A robust and integrated development environment should support automation of the development process.

The environment should include:

1. **Requirements management**
2. **Document automation**
3. **Host/target programming tools**

4. **Automated regression testing**
5. **Continuous and integrated change management**
6. **Feature/defect tracking**

Important Principle

A common characteristic of successful software projects is:

Good people + Good tools → Effective software development

Automation of the design process provides benefits in:

- Quality
- Cost estimation
- Schedule estimation
- Overall productivity
- Ability to work with a smaller team

3.4.4 INTEGRATED TOOLSETS

Integrated toolsets are increasingly important in **incremental/iterative development**.

They allow designers to:

- Move quickly among different development artifacts.
- Keep artifacts up-to-date.
- Maintain consistency between artifacts.
- Automate repetitive activities.
- Support rapid iterations.

Thus:

Integrated tools → Faster artifact transitions → Better iterative development

3.4.5 ROUND-TRIP ENGINEERING

Round-trip engineering is a key capability of environments that support iterative development.

As different information repositories are maintained for engineering artifacts, automation is required to ensure an **efficient and error-free transition of data from one artifact to another**.

Round-trip engineering involves both:

1. Forward Engineering

Forward engineering is the automation of one engineering artifact from another, more abstract representation.

Example:

Source code → Compiler/Linker → Executable code

Compilers and linkers provide automated transition from source code into executable code.

2. Reverse Engineering

Reverse engineering is the generation or modification of a more abstract representation from an existing artifact.

Example:

Source code → Visual design model

A visual design model can be created from an existing source-code representation.

3. Round-Trip Engineering

Round-trip engineering provides environment support to:

- Change an artifact freely.
- Automatically update other related artifacts.
- Maintain consistency among the complete set of artifacts.

These artifacts include:

Requirements → Design → Implementation → Deployment

Basic Representation

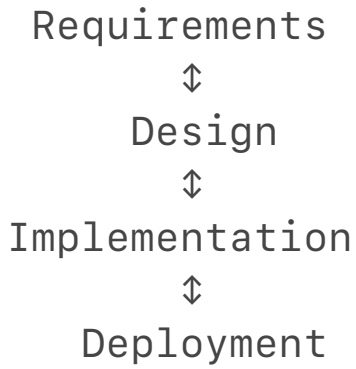


Figure 3.4 – Concept of Round-Trip Engineering

The main objective is **consistency among the entire set of engineering artifacts**.

3.4.6 CONFIGURATION CONTROL AND BUILD MANAGEMENT

As software architectures began using:

- Heterogeneous components
- Different platforms
- Multiple programming languages

the complexity of building, controlling, and maintaining large-scale systems increased.

This created new requirements for:

- **Configuration control**
- **Build management**
- **Automation**

Modern software environments therefore need to automate the management and construction of large systems consisting of many components.

Limitation

The reference points out that today's environments **do not yet provide automation to the full extent possible**.

For example, automated generation of test cases from test-case and scenario descriptions has not evolved much beyond relatively simple cases such as **unit-test scenarios**.

3.4.7 ECONOMIC IMPACT OF TOOLS AND ENVIRONMENTS

A caution is necessary when evaluating the economic benefits claimed for software tools and

environments.
Tool vendors may make individually accurate claims about the percentage of life-cycle activity consumed by their particular area.
However, taking all such claims together can produce a **misleading conclusion**.

Examples of Vendor Claims

According to the reference, individual claims may include:

Software activity	Claimed resource impact
Requirements analysis and evolution	About 40% of life-cycle costs
Software design	More than 50% of resources
Coding and unit testing	About 50% of development effort and schedule
Testing	Up to 50% of project resources
Configuration control and change management	Up to 25% of resources on a large-scale project
Documentation	More than 30% of project engineering resources
Project management, business administration and progress assessment	Up to 30% of project budgets

Important Observation

Individually, these claims may not necessarily be wrong.
But **adding all of them together is misleading**, because the same project resources and activities are being considered from different perspectives.
Therefore:
Do not estimate the total economic impact of a tool by simply adding the percentage claims associated with individual activities.

3.4.8 AUTOMATION AND MODERN SOFTWARE PRACTICES

The modern software practices discussed in the reference have an impact beyond simple process efficiency.
They also contribute to:

- Improved quality
- Reduced development effort
- Better productivity

- Faster iterations
- Better control of evolving artifacts
- More predictable development

Many of the practices discussed in the chapter therefore provide **both efficiency improvement and quality improvement**.

⭐ EXAM FOCUS

Remember the main points

Software Environment → Automation → Productivity + Quality

Important terms

- **Software development environment**
- **Integrated toolset**
- **Configuration management**
- **Process automation**
- **Forward engineering**
- **Reverse engineering**
- **Round-trip engineering**
- **Build management**
- **Regression testing**

Most important concept

Round-trip engineering:

Change one engineering artifact → related artifacts are automatically updated → consistency is maintained.

Important figure

Figure 3.4 – Round-Trip Engineering

`Requirements ⇔ Design ⇔ Implementation ⇔ Deployment`

Important numerical fact

Isolated impact of tools and automation: approximately 20%–40% improvement in effort.

3.5 ACHIEVING REQUIRED QUALITY

Software best practices not only improve **efficiency and productivity**, but also have a major effect on **software quality**.

Many modern software practices provide **better quality for the same cost**, while some provide both **quality improvement and cost reduction**.

The major quality improvements obtained by using a modern software process are summarized below.

Table 3-5: General Quality Improvements with a Modern Process

QUALITY DRIVER	CONVENTIONAL	MODERN ITERATIVE

	PROCESS	PROCESSES
Requirements misunderstanding	Discovered late	Resolved early
Development risk	Unknown until late	Understood and resolved early
Commercial components	Mostly unavailable	Still a quality driver, but trade-offs must be resolved early in the life cycle
Change management	Late in life cycle, chaotic and malignant	Early in life cycle, straightforward and benign
Design errors	Discovered late	Resolved early
Automation	Mostly error-prone manual procedures	Mostly automated, error-free evolution of artifacts
Resource adequacy	Unpredictable	Predictable
Schedules	Overconstrained	Tunable to quality, performance, and technology
Target performance	Paper-based analysis or separate simulation	Executing prototypes, early performance feedback, quantitative understanding
Software process rigor	Document-based	Managed, measured, and tool-supported

Important Observation

The modern iterative process attempts to **identify and resolve problems early**, rather than discovering them during integration and testing.

This reduces:

- Late rework

- Development risk
- Design errors
- Requirement misunderstandings
- Performance problems

KEY PRACTICES THAT IMPROVE SOFTWARE QUALITY

The following practices improve the overall quality of software:

1. Focus on Requirements and Critical Use Cases Early

The development process should focus on:

- Driving requirements early in the life cycle.
- Identifying and implementing **critical use cases** early.
- Ensuring requirements completeness and traceability during the later stages.
- Continuously balancing:
  - Requirements evolution
  - Design evolution
  - Plan evolution

This helps the development team discover misunderstandings and risks at an early stage.

2. Use Metrics and Indicators

Metrics and indicators should be used to measure:

- Project progress
- Quality of the architecture
- Evolution of the system

The architecture should be evaluated as it evolves from:

High-level prototype → Fully compliant product

This provides objective information about whether the project is progressing toward the required quality.

3. Provide Integrated Life-Cycle Environments

The development environment should support:

- Early and continuous configuration control
- Change management
- Rigorous design methods
- Document automation
- Regression test automation

An integrated environment helps maintain consistency among different engineering artifacts and reduces errors caused by manual activities.

4. Use Visual Modeling and Higher-Level Languages

Visual modeling and **higher-level programming languages** support:

- Architectural control
- Abstraction

- Reliable programming
- Reuse
- Self-documentation

These techniques help developers understand and control complex software systems.

5. Use Demonstration-Based Evaluations

There should be **early and continuous insight into performance issues** through:
Demonstration-based evaluations
Instead of depending only on paper-based analysis or simulations, executing prototypes provides realistic feedback about the actual behavior of the system.

PERFORMANCE AS A QUALITY DRIVER

Performance becomes particularly important when a project combines:

- Commercial components
- Custom-developed components

In a conventional software process, considerable effort is placed on making early estimates of computer resource utilization.
However, these early estimates can be inaccurate because many factors are difficult to understand before the system is actually executed.

Typical Chronology of Performance Problems

1. Project Inception

At the beginning of the project, the proposed design is usually considered:

- Low risk
- Capable of meeting performance requirements
- Having adequate performance margin

The performance assumptions are generally optimistic.

2. Initial Design Review

During the initial design review:

- Performance margin is estimated.
- Paper-based analysis or rough simulation may be used.
- Application algorithms are reasonably understood.
- Database sizes may be reasonably understood.

However, several infrastructure-related factors may not be properly understood, such as:

- Operating-system overhead
- DBMS overhead
- Interprocess communication
- Network communication
- Secondary processing threads

Therefore, the initial performance estimate may be overly optimistic.

3. Mid-Life-Cycle Design Review

As development progresses:

- Early benchmarks are performed.
- Initial tests provide actual performance information.
- The original performance margin begins to **erode**.

The team starts discovering that the original assumptions were not completely accurate.

4. Integration and Test

During integration and testing:

- Serious performance problems may be discovered.
- Fundamental architectural changes may become necessary.

Infrastructure is often blamed for the performance problem.

However, the actual causes may be:

- Immature use of infrastructure
- Immature architectural solutions
- Poorly understood design trade-offs during the early stages

WHY CONVENTIONAL PERFORMANCE ANALYSIS FAILS

The above sequence occurs because early performance insight is often based mainly on:

Naive engineering judgment of innumerable criteria

In a large-scale distributed system, many components interact with one another.

Therefore, it is difficult to accurately predict the final performance only through:

- Paper analysis
- Rough simulations
- Engineering judgment

This problem becomes more serious when the system contains many interacting components.

DEMONSTRATION-BASED PERFORMANCE ASSESSMENT

For large-scale distributed systems, a **demonstration-based approach** provides a significantly more accurate assessment of performance.

Instead of waiting until the final stages, the team should execute realistic demonstrations as early as possible.

Early demonstrations may be performed using:

- Host platforms
- Target platforms
- Partial network configurations

These demonstrations should be deliberately **planned and managed as engineering exercises**.

Advantages of Early Demonstration

Early demonstrations help to:

1. Identify performance problems early.
2. Provide realistic performance feedback.

3. Expose architectural weaknesses.
4. Evaluate commercial components.
5. Understand actual system behavior.
6. Allow appropriate trade-offs to be made early.
7. Avoid expensive late-life-cycle architectural changes.

IMPORTANT POINT

Early performance problems are **not necessarily bad**.
They can actually be **healthy for the project** because they expose:

- Architectural flaws
- Architectural weaknesses
- Problems with commercial components
- Incorrect design assumptions

early enough for the development team to make the necessary trade-offs.

3.5 ACHIEVING REQUIRED QUALITY — EXAM SUMMARY

Main Idea

A modern iterative software process improves quality by **discovering and resolving requirements, design, risk, change, resource and performance problems early in the life cycle**.

Important practices:

Requirements → Metrics → Integrated Environment → Visual Modeling & Higher-Level Languages → Demonstration-Based Evaluation

Most Important Point

Early and continuous feedback is the key to achieving required quality.

3.6 PEER INSPECTIONS: A PRAGMATIC VIEW

Peer inspections are often considered an important part of a software quality system. However, the reference material takes a **pragmatic view**: peer reviews are valuable, but they should be treated as **secondary quality mechanisms**, rather than the primary mechanism for achieving software quality.

PRIMARY QUALITY MECHANISMS

According to the reference, the major contributors to software quality are:

1. Transitioning Engineering Information

Engineering information must be transitioned from one artifact set to another while assessing:

- Consistency
- Feasibility
- Understandability
- Technology constraints

This helps ensure that information remains correct as the software evolves.

2. Major Milestone Demonstrations

Major milestones should include demonstrations against **tangible criteria** using relevant use cases.
This provides objective evidence of the system's progress and quality.

3. Environment Tools

Development environments provide important quality mechanisms through tools such as:

- Compilers
- Debuggers
- Analysis tools
- Automated test suites

These tools help ensure:

- Representation rigor
- Consistency
- Completeness
- Change control

4. Life-Cycle Testing

Testing throughout the life cycle helps evaluate:

- Critical trade-offs
- Acceptance criteria
- Requirements compliance

Testing should not be postponed until the end of the project.

5. Change Management Metrics

Change-management metrics provide objective insight into:

- Change trends
- Project progress
- Quality trends
- Convergence or divergence from quality goals

WHEN PEER INSPECTIONS PROVIDE SIGNIFICANT VALUE

Although inspections should not dominate the quality process, they can provide significant benefits in certain situations.

1. Professional Development

Peer inspection can be particularly valuable for **junior engineers**.
A junior engineer's work can be reviewed by senior mentors to:

- Accelerate knowledge and skill development
- Detect mistakes

- Prevent bad practices
- Improve engineering practices

Junior engineers can also learn by inspecting the work of senior engineers.

2. Accountability for Quality

All authors should have their software and documentation scrutinized.
The purpose is to ensure that responsibility for quality is distributed across the team.
Inspection coverage should therefore focus on **authors rather than only components**.
For example:

- Junior authors can have their components inspected periodically.
- Junior engineers can learn by inspecting senior engineers' products.
- Informal inspection can occur continuously during reading, integration, and testing.

3. Critical Components

Critical components should be inspected by several people, particularly stakeholders who have a direct interest in:

- Quality
- Performance
- Feature set

This provides multiple perspectives on important parts of the system.

4. Resolving Existing Issues

An inspection can also be useful when there is already a known problem.
A focused inspection can help:

- Determine the cause of the problem.
- Understand the issue.
- Identify an appropriate resolution.

OVEREMPHASIS ON FORMAL INSPECTIONS

Formal inspections can become **counterproductive** when too much importance is given to them.
Only about **20% of technical artifacts** deserve the level of detailed scrutiny associated with formal inspections.
Examples of technical artifacts include:

- Use cases
- Design models
- Source code
- Test cases

Therefore, a process in which **inspections are the primary quality-assurance activity is not cost-effective**.

LIMITATIONS OF PEER INSPECTIONS

Significant design and architectural errors are often not obvious from a superficial review.

Most inspections tend to be relatively superficial because complex systems are difficult for humans to completely understand.
Random inspections may therefore focus mainly on:

- Coding style
- First-order semantic issues
- Minor defects

They may fail to identify serious architectural problems such as:

- Performance bottlenecks
- Deadlocks
- Race conditions
- Resource contention
- Scalability weaknesses
- Reliability weaknesses
- Interoperability weaknesses

HOW ARCHITECTURAL ISSUES ARE EXPOSED

Architectural problems are better exposed through **rigorous engineering activities**.
The process includes:
Analysis
↓
Prototyping
↓
Experimentation
↓
Constructing design models
↓
Committing the current design model to executable implementation
↓
Demonstrating implementation strengths and weaknesses
↓
Using critical use cases and scenarios
↓
Incorporating lessons learned back into models, use cases, implementations and plans
This provides much stronger evidence about the actual quality of the architecture than a superficial inspection alone.

ROLE OF HUMAN REVIEW

Human reviews and inspections should therefore focus on **critical issues**.
However, they should **not be treated as the primary checkpoints** for software quality.
Early life-cycle artifacts are generally more dependent on subjective human review than later artifacts.
Therefore:

Allocating a large percentage of project resources to human inspections is a bad practice.

QUALITY ASSURANCE IS EVERYONE'S RESPONSIBILITY

Quality assurance should be:

- An integral part of the software process
- The responsibility of the entire engineering team
- Integrated with normal engineering activities

It should **not be treated as a completely separate discipline**.

The evolving engineering baselines should be evaluated by an engineering team that is independent of the architecture and development teams.

The assessment should include:

- Change management
- Trend analysis
- Testing
- Inspection

⭐ 3.6 PEER INSPECTIONS: A PRAGMATIC VIEW — EXAM SUMMARY

Main Idea

Peer inspections are useful for **professional development, accountability, critical components, and resolving specific issues**, but they should not be the primary mechanism for software quality assurance.

Primary quality mechanisms:

Engineering artifact transitions + Milestone demonstrations + Environment tools + Life-cycle testing + Change-management metrics

Key limitation:

Only about **20% of technical artifacts** deserve detailed formal inspection.

Important conclusion:

Quality assurance is everyone's responsibility and should be an integral part of the software process, not a separate discipline.